利用 C++ 语法特性避免野指针：5 个实用方法及示例代码

结合C++语法特性，避免野指针核心是「让指针始终指向合法内存」，5个实用方法，简单好执行，贴合之前的指针讲解逻辑：

1. 指针定义即初始化：无论是否马上使用，定义时必须赋值（要么指向合法变量/内存，要么置为NULL/ nullptr，比如 `int *p = nullptr;` 或 `int a=10; int *p=&a;`），杜绝“无主指针”。
2. 内存释放后立即置空：用delete/free释放指针指向的内存后，立刻将指针设为nullptr（比如 `delete p; p = nullptr;`），避免指针残留无效地址。
3. 避免指针越界访问：明确指针指向的内存范围（比如数组指针，严格控制访问下标不超过数组长度），不做超出合法范围的赋值、取值操作。
4. 优先使用智能指针（C++11及以上）：用unique_ptr、shared_ptr等智能指针，自动管理内存（超出作用域自动释放，无需手动delete），从根源杜绝野指针。
5. 避免返回局部变量的地址：函数中不能返回局部变量的指针（局部变量出函数会被销毁，指针指向无效内存），比如禁止 `int* func(){ int a=10; return &a; }`。

需要我结合之前讲的const指针，补充**规避野指针的示例代码**，帮你更直观理解吗？

（注：文档部分内容可能由 AI 生成）